Semantic MapReduce: Orchestrating LLMs over Large-Scale Data

Anonymous Author(s)

Abstract

Large-scale data systems make it routine to scan, join, window, and aggregate massive observations, but they leave a different operation largely implicit: reducing distributed evidence into auditable semantic conclusions. Directly attaching an LLM to a database or dashboard hides the partitioning, evidence selection, hypothesis merging, and provenance decisions that make such conclusions reproducible. This paper formulates *Semantic MapReduce*: a systems contract for sharding observations, extracting bounded local evidence, normalizing and grouping evidence objects, reducing them into incident hypotheses, and generating evidence-linked workflow traces. The abstraction positions an AI-native orchestration layer above existing data engines and model-serving systems rather than replacing them. We instantiate the contract in an anonymized workflow/stream prototype and evaluate it with a reproducible NPU-backed LLM-serving telemetry workload plus a semantic-merge suite. Across a 10-seed matrix and two workload sizes, the implemented incident reducer reaches mean F1 0.9579 with the tail-aware evidence policy, compared with 0.3075 F1 for a map-only alert baseline and 0.7129 for a window-aggregation baseline. The semantic-merge suite separates sanity, cascade, shared-bottleneck, concurrent, false-correlation, partial-evidence, and ambiguous-overmerge cases; a graph-aware semantic reducer reaches 0.7696 mean F1, and a hint-aware hybrid reducer raises this to 0.8173 by repairing missing-root evidence cases. Live LLM probes show why raw prompting is not a sufficient reducer interface: free-form JSON edits can be malformed or unsafe, while a one-token pairwise action interface lets the system assemble validated edits and improves a covered three-hardcase probe from 0.6948 to 0.8857 mean F1 with zero invalid actions, invalid schemas, or fallbacks. The prototype addresses this challenge with evidence schemas, deterministic baselines, coverage gates, constrained validated edits, token accounting, and auditable traces.

1 Introduction

The dominant abstraction in large-scale analytics separates local data processing from global aggregation. MapReduce made this pattern explicit with map tasks and reducers [3]; Spark and Flink extended it to richer batch, iterative, and stream computations [2, 19]; Ray generalized distributed execution for AI and Python workloads [10]. These systems remain the right substrate for moving records, executing scans, joining tables, maintaining windows, and scheduling distributed work.

LLM-augmented analysis stresses a different part of the stack. In operational settings, the desired output is often not another aggregate but an interpretation: which symptoms are part of the same incident, which evidence supports the hypothesis, which observations conflict with it, and what action should an operator take next. The hidden assumption in conventional analytics is that the global operation is a numeric or relational aggregation. For LLM-native analysis, the global operation is often a *semantic reduction* over distributed evidence.

A direct workaround is to place an LLM on top of a database, log system, dashboard, or agent tool loop. This is useful for exploration, but it leaves the semantic systems contract undefined. The model context is too small for raw telemetry, prompt state is hard to audit, local evidence is selected by application glue code, and final answers often do not preserve the exact workflow decisions that produced them. Conversely, replacing Spark, Flink, Ray, databases, or model-serving systems with a new monolithic LLM system would duplicate mature execution machinery and obscure the layer at which the semantic problem actually appears.

This paper studies the orchestration-layer problem: how should a system coordinate LLM-augmented analysis when observations are large, partitioned, and heterogeneous, but the desired result is an auditable semantic conclusion? The key design constraint is that raw data movement and model execution should remain in specialized engines. The orchestration layer should instead own the contract that turns partitions into evidence, evidence into grouped candidates, candidates into hypotheses, and hypotheses into traceable reports.

We call this contract *Semantic MapReduce*. Like MapReduce, it separates local processing from global reduction. Unlike MapReduce, its intermediate data are evidence objects rather than arbitrary key-value pairs, and its reducer emits hypotheses with provenance rather than only sums, counts, joins, or window aggregates. The standard operators are Shard, MapEvidence, Normalize, GroupEvidence, SemanticReduce, and ReportTrace. These operators are intentionally small: they define where existing data engines, LLM workflow frameworks, serving endpoints, and audit consumers can connect without collapsing the whole analysis into one prompt or one script.

Figure 1 illustrates the difference. A naive stitched pipeline can connect telemetry, scripts, an agent, and a report, but the incident boundary remains implicit: are repeated shard-local alerts the same failure, or independent failures? In our

NPU-backed LLM-serving telemetry workload, map-only alerts reach 0.3075 mean F1 and emit 3.50 outputs per true incident; window aggregation improves F1 to 0.7129 but still emits 1.76 fragments per incident. The implemented incident reducer reaches 0.9579 F1 and reduces the report load to 1.04 outputs per true incident. A trace-guided baseline-aware evidence policy later recovers low-baseline latency incidents that tail-aware evidence can miss. This result isolates the systems contribution: the missing operation is incident-level semantic reduction over evidence objects, coupled with auditable evidence policies that reveal which operator boundary needs repair.

We instantiate Semantic MapReduce in an anonymized workflow/stream prototype. The prototype is not presented as a new data engine. It exposes an operator graph, stream composition, reducer plug-ins, evidence objects, and workflow traces while keeping storage engines and LLM serving endpoints behind integration boundaries. This separation lets the same workload run with a map-only diagnostic baseline, a window-aggregation baseline, a deterministic incident reducer, an `llm-stub` interface check, and a live OpenAI-compatible reducer path without changing the generator or scorer.

The evidence boundary is deliberately narrow because Semantic MapReduce makes LLM-backed reduction a systems problem rather than a prompt-only problem. The main quality results use deterministic and hybrid reducers so that the workload, scorer, and trace semantics remain reproducible. The `llm-stub` reducer intentionally matches the deterministic baseline because it is an offline interface check. Single-NPU online runs then stress the live reducer contract: a model may omit incidents, over-merge candidates, produce invalid structure, or add token latency. The prototype turns these behaviors into explicit mechanisms: structured evidence, coverage gates, constrained candidate edits, schema and evidence validators, no-regression fallback, and cost/latency accounting. In the hardest live probes, the system narrows model output from free-form reducer JSON to one enumerated action per candidate pair, then constructs and validates the edit itself.

This paper makes four contributions:

1. It defines Semantic MapReduce as an operator algebra for LLM-native analysis over partitioned observations: Shard, MapEvidence, Normalize, GroupEvidence, SemanticReduce, Edit, Validate, and ReportTrace.
2. It maps the contract onto workflow, stream, runtime, evidence, reducer, trace, and serving-integration boundaries, showing how an AI-native orchestration layer can sit above existing data and execution engines.
3. It introduces reproducible workload families for NPU-backed LLM-serving telemetry and semantic incident merging, and records a pinned shared LLM-serving workload probe for scenario provenance; the mechanism workloads include injected incidents, evidence operators, reducer variants, precision/recall/F1 scoring, and trace artifacts.
4. It reports a 10-seed evaluation, trace-guided evidence-policy ablations, substrate probes, and single-accelerator online LLM-reducer comparisons, showing how coverage gates, constrained candidate validation, and cost accounting make LLM-reducer behavior inspectable under the same schema and scorer.

The rest of the paper is organized as follows. Section 2 motivates the orchestration-layer gap. Section 3 defines the problem. Section 4 formalizes the Semantic MapReduce operator model. Section 5 presents the workflow/stream design. Section 6 describes the workload. Section 7 evaluates reducer quality, evidence-policy ablations, substrate checks, and online readiness. Sections 8–10 discuss open system challenges, related systems, and conclusions.

2 Motivation

Operational analysis often starts with numerical aggregates but ends with semantic judgment. In an accelerator-backed LLM serving platform, a dashboard may show that p95 latency increased in one region, decode utilization saturated, scheduler queue depth grew, and errors rose for some tenants. Each observation is measurable with conventional analytics; the harder question is whether these observations are one incident, several unrelated incidents, or noise. Conventional engines provide the substrate for scans, windows, joins, and stateful aggregation, but they do not normally expose first-class incident hypotheses, evidence objects, conflict handling, or explanation traces.

LLM-enabled data tools often expose a chat interface over a database, log store, vector index, or dashboard. This is useful for ad hoc exploration, but it hides important systems structure: which context was inspected, how evidence was compressed, when follow-up queries were issued, and why a final answer should be trusted. Large-scale analysis needs explicit partitioning, bounded local evidence, semantic reduction, and traceable reports.

The orchestration layer is the right place for this contract. It does not replace Spark, Flink, Ray, SQL engines, vector stores, or serving engines. Those systems own distributed execution, storage access, stream processing, indexing, model serving, and resource management. The orchestration layer expresses the LLM reasoning workflow as a dataflow/stream pipeline, coordinates semantic operators, and records intermediate evidence.

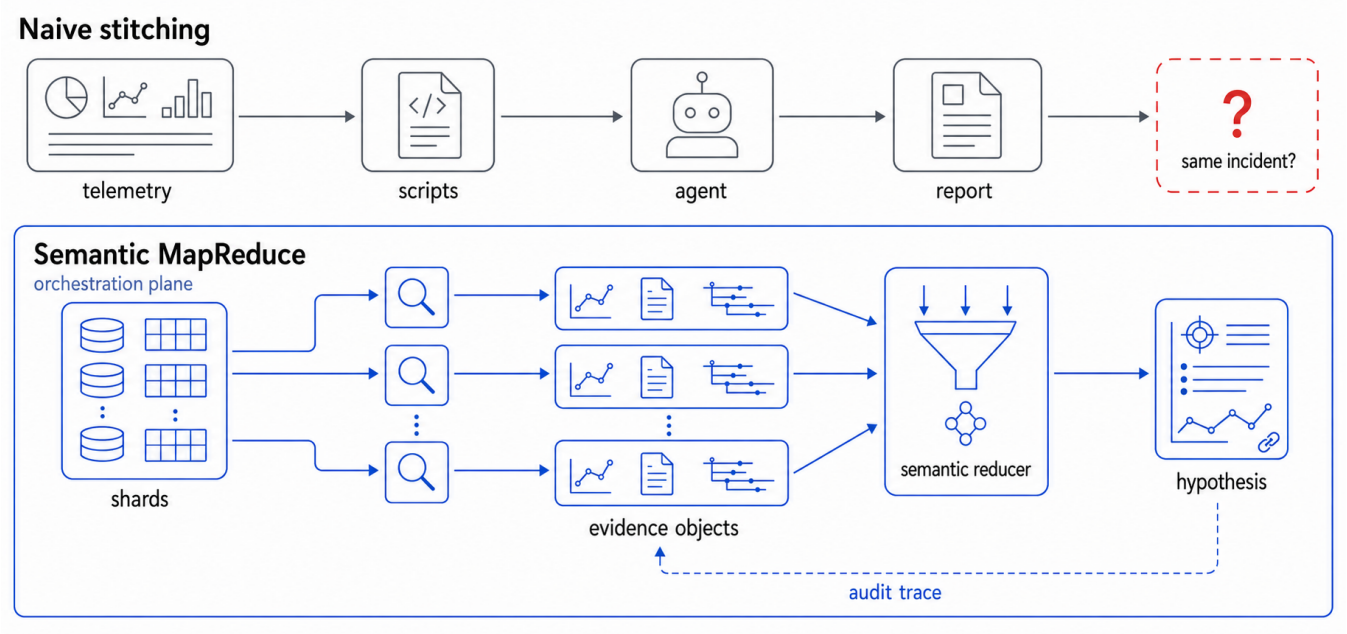

Figure 1. Motivating comparison. Naive stitching can connect telemetry, scripts, and an LLM agent, but it leaves the central systems question implicit: which shard-local symptoms should be merged into the same incident? Semantic MapReduce makes that boundary explicit. Shards produce evidence objects, a semantic reducer performs incident-level fusion, and the report carries audit links back to the evidence that supports each hypothesis.

3 Problem Statement

We define large-scale LLM-augmented data analysis as follows. The input is a large collection of heterogeneous observations: events, logs, metrics, traces, documents, experiment outputs, and derived aggregates. The output is a set of structured insights, incident hypotheses, explanations, evidence chains, and possible actions. A correct system must manage both scale and semantics.

This problem exposes six system challenges:

- **Scale and partitioning.** Data must be split across shards while preserving enough local context to extract meaningful evidence.
- **Evidence compression.** Raw records cannot all be sent to a reducer or LLM. Map stages must summarize without losing the signals needed for global reasoning.
- **Semantic reduction.** The reducer must merge related symptoms, remove duplicates, handle conflicts, and produce hypotheses rather than only numeric aggregates.
- **Auditability.** Reports must link back to the evidence and workflow decisions that produced them.
- **Latency and cost.** LLM calls, if used, must be bounded and placed where semantic value justifies cost.
- **Safety boundaries.** The orchestration layer must avoid leaking unnecessary raw data and must make it possible to restrict what reaches external models.

These constraints motivate a system that treats LLM interpretation as a workflow over evidence objects, not as a final chat step after data processing.

4 Semantic MapReduce Operator Model

The central claim of Semantic MapReduce is that LLM-native analysis needs standard operators, analogous in spirit to `map` and `reduce`, but with contracts for evidence, bounded semantic edits, validation, and explanation. The abstraction is not an agent prompt wrapped around a database. It is an operator model in which observations become evidence, evidence becomes reducer candidates, reducer candidates become hypotheses, and every semantic edit is either validated or rejected before it affects the report.

Let D denote a collection of observations, $\mathcal{E}$ evidence objects, C reducer candidates, $\mathcal{H}$ hypotheses, Δ reducer edits, $\mathcal{T}$ workflow trace records, and $\mathcal{Y}$ the final report. Semantic MapReduce composes the following operators:

$$\mathcal{A} = \{\textsc{Keep}, \textsc{Merge}, \textsc{Split}, \textsc{Abstain}\} \tag{1}$$

$$\mathcal{S} = \mathsf{Shard}(D; \pi) \tag{2}$$

$$\mathcal{E} = \bigcup_{s \in \mathcal{S}} \mathsf{MapEvidence}(s; \phi) \tag{3}$$

$$\widehat{\mathcal{E}} = \mathsf{Normalize}(\mathcal{E}; \sigma) \tag{4}$$

$$\mathcal{C} = \mathsf{GroupEvidence}(\widehat{\mathcal{E}}; \gamma) \tag{5}$$

$$(\mathcal{H}_0, \mathcal{T}_R) = \mathsf{SemanticReduce}(\mathcal{C}; \rho) \tag{6}$$

$$\Delta = \mathsf{Edit}(\mathcal{C}, \mathcal{H}_0; a), \quad a \in \mathcal{A} \tag{7}$$

$$(\mathcal{H}, \mathcal{T}_V) = \mathsf{Validate}(\mathcal{H}_0, \Delta, \widehat{\mathcal{E}}; V) \tag{8}$$

$$(\mathcal{Y}, \mathcal{T}) = \mathsf{ReportTrace}(\mathcal{H}, \widehat{\mathcal{E}}, \mathcal{T}_R \cup \mathcal{T}_V; \tau). \tag{9}$$

The policies $\pi, \phi, \sigma, \gamma, \rho, V, \tau$ may be implemented by rules, queries, stream processors, graph algorithms, LLM calls, or hybrids. The important invariant is that only structured objects cross operator boundaries. Local findings are evidence objects rather than prompt text; reducer inputs are candidate groups rather than raw telemetry; model-backed edits are bounded actions rather than complete incident reports; and reports preserve evidence links rather than only natural-language conclusions.

This model separates the part a model may do from the part the system must own. SemanticReduce can be deterministic, graph-based, LLM-backed, or hybrid. When a model is used, the safer interface is often Edit: the model makes a bounded semantic decision such as whether two candidates should be merged, split, kept separate, or abstained from. The system then assembles the edit payload, binds evidence identifiers, checks the incident schema, verifies root and affected-service consistency, and falls back when validation fails. This is why the evaluated one-token reducer is not merely a prompt variant: it changes the reducer contract from "generate JSON hypotheses" to "choose a bounded action that the system can validate."

External data engines may implement the scale-facing operators, while LLM frameworks may implement semantic decisions or reporting. The orchestration layer composes them under one auditable evidence contract and owns the boundaries where data-plane outputs become semantic-control-plane decisions. The operators expose five contract surfaces: data engines provide shards, windows, summaries, or chunks; local findings become bounded evidence objects rather than prompt text; reducers may change strategy while preserving the incident schema and scorer; model-backed edits must pass evidence and schema validation; and ReportTrace ties every hypothesis to operator decisions, supporting evidence, and replay metadata.

Shard and map evidence. Each shard is processed locally. MapEvidence may compute statistics, identify anomalies, summarize logs, classify symptoms, or invoke an LLM over a bounded context. Its output is not arbitrary prose. It is an evidence object with fields such as service, tenant, region, metric, time window, observed value, baseline, severity, confidence, and source references.

Normalize, group, and semantic reduce. Normalize and GroupEvidence prepare evidence for global reasoning. SemanticReduce then merges evidence objects into higher-level hypotheses. This stage deduplicates adjacent windows, combines weak but consistent signals, separates unrelated incidents, handles contradictory evidence, and assigns confidence. The reducer may be deterministic, LLM-backed, or hybrid. Deterministic reducers are useful baselines because they are reproducible and cheap; LLM reducers are attractive when the merge requires domain language, causal reasoning, or flexible explanation.

Edit and validate. A model-backed reducer need not emit final hypotheses. It can instead implement Edit: a bounded decision over reducer candidates. In the evaluated action reducer, the model chooses only Keep, Merge, Split, or Abstain. Validate then checks that any proposed edit preserves evidence references, schema constraints, root and affected-service consistency, and the no-regression fallback policy. Invalid model text becomes Abstain; invalid edits are rejected; unsafe reducer outputs fall back to the deterministic or hybrid baseline.

Report and trace. ReportTrace turns hypotheses into operator-facing reports. The report should include the claim, supporting evidence, possible conflicting evidence, and suggested actions. It should also preserve the workflow trace so a user can inspect how the conclusion was formed.

Semantic MapReduce differs from an agent loop because its units of parallelism, evidence compression, reduction, and reporting are explicit. It also differs from ordinary stream processing: stream windows can compute features and alerts, but semantic reduction turns local findings into a global explanation.

Algorithm 1 makes the operator vocabulary operational. The important constraint is that SemanticReduce consumes compact, normalized, grouped evidence objects rather than raw telemetry, and that model-backed Edit decisions are validated before they affect $\mathcal{H}$. This lets the orchestration layer place expensive or policy-sensitive LLM calls at semantic boundaries instead of at every scan or filter operation.

5 Runtime and Orchestration Design

The prototype keeps the execution boundary deliberately narrow. Storage, joins, stream windows, distributed scheduling, and model serving remain delegated to existing engines; the orchestration layer owns the semantic control plane: operator composition, evidence schemas, reducer selection, validation, and trace capture. This is the difference from simple stitching. A script can query a data engine and then call a model, but it does not define which evidence objects cross

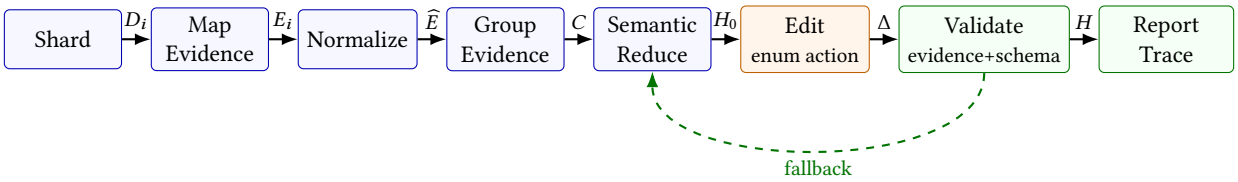

Figure 2. Operator-algebra view of Semantic MapReduce. The model-facing boundary is Edit, not full report generation: an LLM may choose a bounded action over candidates, while Validate remains system-owned and enforces evidence references, schema constraints, root/affected consistency, and fallback before ReportTrace.

Algorithm 1 Semantic MapReduce Workflow

Require: Observations D, policies $\pi, \phi, \sigma, \gamma, \rho, V, \tau$
Ensure: Evidence-linked incident report $\mathcal{Y}$ and workflow trace $\mathcal{T}$

$\mathcal{S} \leftarrow \textsc{Shard}(D; \pi)$ ▷ partition by service, region, tenant, or time
$\mathcal{E} \leftarrow \emptyset, \mathcal{T} \leftarrow \emptyset$
for all shard $s \in \mathcal{S}$ **do**
  $(E_s, T_s) \leftarrow \textsc{MapEvidence}(s; \phi)$
  $\mathcal{E} \leftarrow \mathcal{E} \cup E_s$
  $\mathcal{T} \leftarrow \mathcal{T} \cup T_s$
end for
$\widehat{\mathcal{E}} \leftarrow \textsc{Normalize}(\mathcal{E}; \sigma)$
$\mathcal{C} \leftarrow \textsc{GroupEvidence}(\widehat{\mathcal{E}}; \gamma)$
$(\mathcal{H}_0, T_R) \leftarrow \textsc{SemanticReduce}(\mathcal{C}; \rho)$
$(\Delta, T_E) \leftarrow \textsc{Edit}(\mathcal{C}, \mathcal{H}_0)$ ▷ optional bounded edit
$(\mathcal{H}, T_V) \leftarrow \textsc{Validate}(\mathcal{H}_0, \Delta, \widehat{\mathcal{E}}; V)$
$\mathcal{T} \leftarrow \mathcal{T} \cup T_R$
$\mathcal{T} \leftarrow \mathcal{T} \cup T_E \cup T_V$
$\mathcal{Y} \leftarrow \textsc{ReportTrace}(\mathcal{H}, \widehat{\mathcal{E}}, \mathcal{T}; \tau)$
return $(\mathcal{Y}, \mathcal{T})$

the boundary, how duplicate local alerts become one hypothesis, or how a report links back to reducer decisions.

The workflow graph is the runtime representation of this contract. It composes familiar stream/dataflow operators–map, filter, flatmap, key grouping, and connection–but assigns them semantic roles. Sharded maps implement MapEvidence; keyed stages implement GroupEvidence; downstream stages invoke SemanticReduce, optional Edit, Validate, and ReportTrace. Upstream systems may supply partitions, windows, feature tables, retrieved documents, or trace chunks. Downstream tools consume structured incidents and evidence-linked reports. The orchestration layer therefore sits above Spark, Flink, Ray, databases, vector stores, and serving endpoints rather than replacing them.

Evidence objects are the bridge between data systems and LLM reasoning. They include shard and time-window identifiers, service/region/metric fields, local baseline statistics, anomaly type, severity, confidence, and source references. Reducers consume grouped evidence and emit the same incident schema plus a reducer trace. The evaluated implementation includes map-only and window baselines, deterministic and hybrid semantic reducers, free-form LLM reducers, and action-constrained reducers under one scorer. Changing the reducer therefore does not require rewriting the generator, mapper, report schema, or benchmark matrix. The trace records reducer metadata, detected incidents, matched and missed incident IDs, evidence summaries, failure classes, run manifests, and token estimates, so false positives and false negatives can be attributed to evidence extraction, grouping, editing, validation, or thresholding rather than to an opaque final answer.

6 Workload and Provenance

6.1 Scenario

The workload models telemetry from an NPU-backed LLM serving platform. Services include `prefill`, `decode`, `kv-cache`, `scheduler`, `router`, and `embedding`. Events include service, tenant, region, timestamp, latency, error flag, NPU utilization, queue depth, and generated metadata. The generator injects hidden incidents of three types:

- **Latency spike:** one service experiences elevated latency over a time window.
- **NPU saturation:** utilization increases and may affect queueing or latency.
- **Queue backlog:** scheduler or routing queues grow and create downstream symptoms.

The task is to recover injected incidents from event streams. A detected incident is useful only if it links to the supporting evidence and matches the hidden ground truth by type, service, region, and time.

Why this workload exercises the system. The workload is designed to stress the orchestration layer rather than raw storage throughput. It has enough structure to require semantic reasoning: incidents are injected across services, regions, and time windows, while local symptoms appear as latency, error, utilization, or queue-depth changes. A single shard can observe symptoms, but the global conclusion depends on how those symptoms are merged. This matches the Semantic MapReduce pattern.

The workload also forces the system to preserve evidence. If the reducer reports a latency spike, the report must include which shard-level candidates supported the decision. If it misses a saturation incident, the report should expose the missed incident ID. This is why the workload reports both detected and injected incidents, not only aggregate F1. The goal is not merely to score a classifier; it is to evaluate an auditable analysis workflow.

Finally, the workload is intentionally reproducible. Seeds, event counts, shard counts, top-k values, reducer names, incident reports, missed incidents, reducer traces, workflow traces, run manifests, failure classes, and token/cost accounting fields are recorded. The same matrix can be rerun when a reducer changes. This gives the prototype a benchmark harness for model-backed reducer experiments without making the current paper depend on a live model endpoint.

The artifact also includes a semantic-merge suite that separates several workload roles. *Single-service* is a sanity case where local alert-style reducers can recover the incident unit. *Cascade* and *shared-bottleneck* cases model root causes whose symptoms appear in dependent services. *Concurrent* and *false-correlation* cases test whether the reducer over-merges unrelated symptoms that overlap in time. *Partial-evidence* removes direct root-service evidence for some incidents, leaving only downstream evidence and upstream hints. These families keep the evidence schema and scorer fixed while varying the semantic-reduction difficulty.

6.2 Pipeline and Implementation

The workload instantiates the operator model directly. Shard partitions generated telemetry by shard count, service, region, and time. MapEvidence computes shard-level statistics, anomaly candidates, and local summaries. Normalize converts candidates into scored evidence objects with provenance fields. GroupEvidence clusters candidates by incident type, service, region, and temporal overlap. SemanticReduce emits incident hypotheses with map-only, window, deterministic, hybrid, stubbed, or OpenAI-compatible reducers. Edit and Validate are exercised by candidate-level and one-token action reducers: the model may propose bounded keep/merge/split/abstain decisions, while the system assembles evidence-linked edits, checks schema/root/affected-service invariants, and falls back when required. ReportTrace scores hypotheses, records matched/missed incidents, classifies reducer failures, and emits per-run reports.

The workload reports precision, recall, F1, throughput, map latency, reduce latency, detected incidents, injected incidents, and missed incidents. Reports also include seed, top-k, reducer name, and incident matching metadata.

The implementation has three components. The generator creates synthetic telemetry and hidden incidents. The map stage implements Shard, MapEvidence, and Normalize by partitioning events, computing local summaries, and producing anomaly candidates. The reducer implements GroupEvidence and SemanticReduce over shard summaries. The default reducer is deterministic: it clusters candidates using incident type, service, region, and temporal overlap. The map-only and window-aggregation reducers are diagnostic baselines that approximate alert-only and windowed-analytics workflows without incident-level semantic reduction. The `llm-stub` reducer is an interface placeholder that exercises the same resolver and report path while delegating to the deterministic reducer.

The report schema is deliberately richer than the minimum needed for precision and recall. It includes the map policy, reducer name, seed, top-k, injected incidents, detected incidents, matched incident IDs, and missed incidents. This enables per-run diagnosis of operator behavior. When a run misses an incident, the missed incident ID is preserved in the report rather than disappearing into the aggregate recall number; when a map policy changes, the same reducer and scorer can be replayed over the new evidence stream.

A typical run proceeds as follows. The generator emits events for services such as `prefill`, `decode`, and `scheduler`. It injects incidents such as a latency spike or NPU saturation over a bounded time window. The shard map stage computes local evidence: elevated p95 latency, higher queue depth, or high NPU utilization. Each local summary is compact enough to be passed to the reducer. The reducer then merges adjacent or related candidates into incident hypotheses. The final report presents incident type, service, region, time window, confidence-like scores, and matching information.

This walkthrough is simple, but it illustrates the value proposition. The workload is not a single SQL query and not a single prompt. It is a workflow whose operators correspond to system responsibilities: sharding, local evidence extraction, normalization, grouping, semantic merging, reporting, and evaluation. Each responsibility can evolve independently. The implemented OpenAI-compatible reducer can replace the deterministic reducer without changing the generator or scorer; a real telemetry source can replace the synthetic generator without changing the reducer contract.

The workload is designed to make reducer behavior reproducible without placing environment-management details in the paper body. Each experiment records the workload scale, shard count, seed, reducer, top-k budget, detected incidents, missed incidents, and latency measurements. This provenance is sufficient to distinguish algorithmic changes in the reducer from changes in input generation or scoring. The accompanying artifact contains the implementation and per-run records needed to rerun the study.

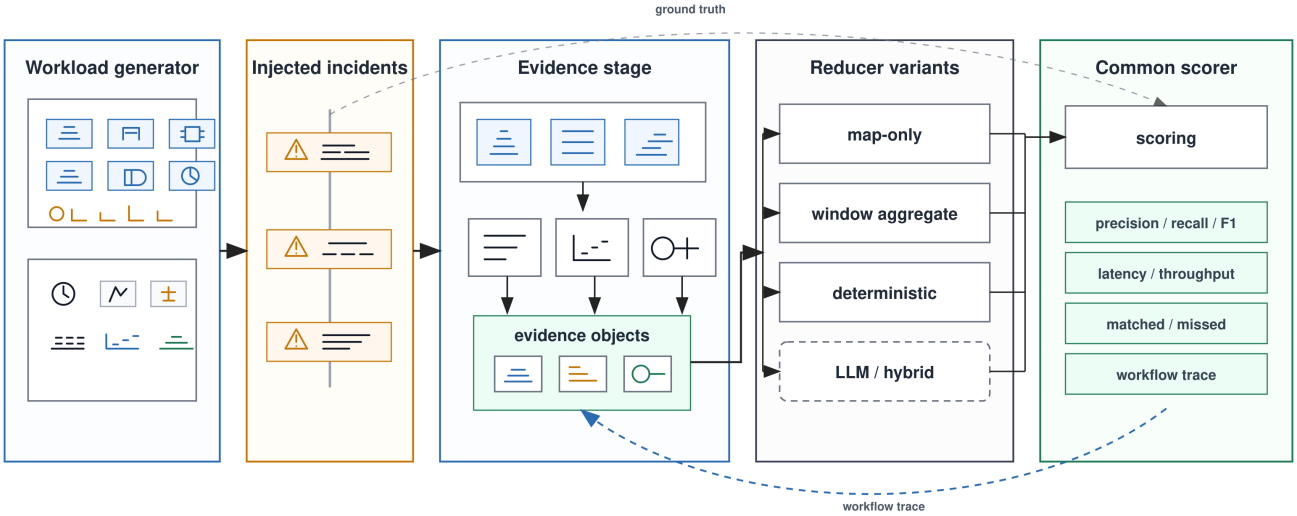

Figure 3. Evaluation harness for the large-scale analysis workload. The same generated telemetry and injected incidents feed the evidence stage, reducer variants, and common scorer. Solid reducer rows correspond to implemented comparison regimes, while the dashed LLM/hybrid branch marks planned extensions rather than completed LLM-reducer results. Ground truth is used only by the scorer, and workflow traces link results back to evidence objects.

Table 1. Workload configurations. Each configuration is run with ten seeds (7, 11, 13, 17, 19, 23, 29, 31, 37, and 41), two map policies, and four reducer regimes.

Events	Shards	Top-k	Runs
50,000	16	12	80
100,000	32	16	80

Table 2. Shared workload-source probe from the pinned workload submodule. This table records scenario coverage and provenance, not Semantic MapReduce reducer quality.

Field	Value
Submodule commit	`79ed8e3`
Seeds	7, 11, 13
Generated cases / seed	23
Skipped boundary cases	2
Supported requests / seed	1,232
Example long-context prompt	4,442 tokens
Example RAG prompt	2,974 tokens
Evidence label	derived artifact

7 Evaluation

7.1 Setup and Scope

Table 1 lists the reproduced configurations. The evaluation is deliberately scoped to operator behavior under one workload harness. The main comparison fixes the generator, evidence schema, scorer, and report format, then varies two pieces: the map evidence policy and the reducer. This separates three questions: whether incident-level reduction helps, whether the map stage preserves enough evidence for any reducer to succeed, and whether adjacent orchestration frameworks provide the missing semantic contract by themselves. The experiments are not tuned cluster deployments of Spark, Flink, Ray, LangGraph, LlamaIndex, or AutoGen.

The artifact also includes a separate shared-workload probe from a pinned workload submodule. Across seeds 7, 11, and 13, the probe generates 23 canonical LLM-serving cases per seed and 1,232 supported requests per seed, covering session-affine, RAG, long-context, tool-agent, coding-assistant, dynamic-corpus, memory-reuse, and preemption/resume surfaces. We use this probe to record workload-source provenance and to keep prototype-specific Semantic MapReduce stress cases separate from general LLM-serving scenario catalogs. It is not used as evidence that a reducer improves incident quality.

Table 2 gives the shared workload side of the artifact. The corresponding Semantic MapReduce quality experiments remain repo-local because they require injected incident labels, evidence objects, reducer variants, and a common scorer. This separation prevents a common benchmark mistake: using a general workload catalog as if it were already a reducer-quality benchmark.

Table 3. Reducer comparison with the tail-aware map policy across 20 runs. Map-only and window-aggregate are diagnostic baselines for alert-style and windowed-analytics workflows. The `llm-stub` row delegates to the deterministic reducer and is included only as an interface check.

Reducer	Precision	Recall	F1	Detections
Map-only	0.1990	0.6875	0.3075	14.00
Window-aggregate	0.5696	0.9750	0.7129	7.05
Incident reducer	0.9500	0.9750	0.9579	4.15
`llm-stub`	0.9500	0.9750	0.9579	4.15

Table 4. Semantic-merge suite across seven workload families and ten seeds. The suite includes a single-service sanity case, cascade and shared-bottleneck incidents, concurrent incidents, false correlations, partial-evidence cases, and ambiguous over-merge cases.

Reducer	Precision	Recall	F1	Detections
Map-only	0.0667	0.1429	0.0906	14.33
Service-local	0.0685	0.1429	0.0922	13.46
Window-aggregate	0.3457	0.6929	0.4531	8.07
Semantic-graph	0.7907	0.7643	0.7696	3.93
Hybrid-hint	0.8367	0.8143	0.8173	3.93
`llm-stub`	0.7907	0.7643	0.7696	3.93

7.2 Reducer Quality

The aggregate results in Table 3 show why incident-level reduction is the relevant operation. Map-only alerts emit duplicate shard-local candidates and consume the top-k budget before all incidents are represented. Window aggregation improves recall but still reports multiple windows for one incident. The incident reducer keeps recall at 0.9750 while reducing average detections from 7.05 to 4.15, raising F1 from 0.7129 to 0.9579. This is the measured advantage in the current workload: semantic reduction changes the unit of output from alert fragments to incident hypotheses.

Table 4 adds the semantic-merge suite. A root cause in one service can produce symptoms in dependent services, so the scorer requires the hypothesis to recover the root service, region, overlapping time, and enough affected services. Unlike a single adversarial case, the suite includes a sanity scenario where map-only reaches 0.6345 F1, confirming that the benchmark is not constructed to make local reducers always fail. On the harder families, the graph-aware semantic reducer reaches 0.9143 F1 on cascades, 0.8667 on shared bottlenecks, 0.7964 on concurrent incidents, and 0.8050 on false correlations. The suite then turns two common semantic-reduction challenges into mechanism tests. In *partial-evidence*, direct root evidence is absent, so a hypothesis may infer the right root but omit it from the affected-service set; the hint-aware hybrid reducer repairs this contract-level error and raises F1 from 0.5761 to 0.8878 without changing the evidence schema or scorer. In *ambiguous-overmerge*, all root evidence is present, but overlapping related incidents collapse into one candidate; this motivates an evidence-bounded split operation rather than another root hint. These cases are not presented as generic model failures. They define the reducer behaviors that a Semantic MapReduce system must surface, constrain, and evaluate.

Table 5. MapEvidence policy ablation for the deterministic incident reducer. The tail-aware policy preserves high-p95 NPU saturation evidence; mean-only corresponds to the earlier mean-utilization rule.

Size	Map policy	Precision	Recall	F1	Missed runs
50K/16/12	Mean-only	0.9200	0.9750	0.9413	1/10
50K/16/12	Tail-aware	0.9200	0.9750	0.9413	1/10
100K/32/16	Mean-only	0.9750	0.9250	0.9464	3/10
100K/32/16	Tail-aware	0.9800	0.9750	0.9746	1/10

Table 6. Coverage gate over ten seeds. A reducer-quality comparison is meaningful only when injected incidents enter the evidence objects. Baseline-aware evidence reaches full coverage for 10K and larger settings in this workload; the 2K tail-aware setting does not.

Config	Policy	Coverage mean	Coverage min	Candidates mean
2K/4/8	Tail-aware	0.3000	0.0000	1.9
2K/4/8	Baseline-aware	0.7750	0.5000	5.1
10K/8/12	Tail-aware	0.9250	0.7500	18.2
10K/8/12	Baseline-aware	1.0000	1.0000	39.5
50K/16/12	Tail-aware	0.9750	0.7500	48.6
50K/16/12	Baseline-aware	1.0000	1.0000	97.7
100K/32/16	Tail-aware	0.9750	0.7500	47.5
100K/32/16	Baseline-aware	1.0000	1.0000	112.0

Table 5 is the most important diagnostic result. The larger workload exposes a failure that is not solved by changing the reducer alone: with mean-only NPU detection, three of ten 100K runs miss at least one incident. The missed cases include NPU saturation windows whose mean utilization is diluted by surrounding normal events, even though their tail utilization is high. Adding a stricter p95 NPU signal to MapEvidence reduces missed runs from three to one and raises 100K F1 from 0.9464 to 0.9746.

Table 6 adds a stronger guardrail: before comparing reducers, the system should measure whether the map stage surfaced the injected incidents as evidence. A later trace-guided pass exposed a second evidence failure: low-baseline services such as routers can suffer real latency spikes whose absolute p95 remains small and whose window mean rises with the incident. A baseline-aware map policy compares each service/region window against a robust shard-local latency baseline. On the same two-size, ten-seed matrix, this policy raises the deterministic incident reducer to 1.0000 mean precision, recall, and F1, while map-only remains at 0.3500 F1 and window aggregation at 0.6366 F1. This result

strengthens the operator argument: once map, evidence, and reduce are separate contracts, a failure can be localized to the evidence policy rather than vaguely attributed to "the LLM" or "the reducer."

The ablation also prevents overclaiming. Tail-aware and baseline-aware evidence policies are workload policies, not universal detectors. Baseline-aware evidence increases the number of map candidates, so it must be paired with incident-level reduction; otherwise map-only precision falls sharply. A stronger system would learn or calibrate the evidence policy from real traces.

7.3 Substrate and Overhead Checks

Local runtime checks show that the map stage dominates because it scans and summarizes event shards, while the deterministic reduce stage runs over compact candidates. Adjacent-framework probes with Ray-local, LangGraph-local, and LlamaIndex-docstore wrappers obtain the same F1 as the native path when they are given the same evidence objects, reducer, scorer, and report format. These probes are not leaderboards over those systems. They support the narrower systems claim that existing substrates can carry the computation, but this workload still needs the evidence-linked reducer contract and audit trail.

7.4 Real-Online Readiness

The real-online runs attach the same reducer contract to a live OpenAI-compatible endpoint. A smoke run served a 7B instruction-tuned model on one accelerator and completed 8/8 measured streaming requests after warmup, with mean TTFT 65.68 ms, mean TPOT 19.79 ms, and 45.71 output tokens/s. These numbers validate endpoint integration and measurement plumbing rather than SOTA serving performance. Earlier same-seed reducer probes exposed two preconditions for meaningful live LLM comparison. First, if map evidence misses incidents, no downstream reducer can recover them: a 2K/4-shard smoke produced F1 0.4000 because three injected incidents had zero overlapping map candidates. Second, asking a model to emit full reducer JSON is the wrong interface: full-evidence prompting returned legal JSON while losing incident units, and free-form candidate editors sometimes produced invalid structure or unsafe edits. The final live probe therefore focuses on the constrained `Edit+Validate` interface in Table 7.

A follow-up pairwise edit probe narrows the interface further. Instead of asking the model to rewrite complete candidates, the reducer proposes short candidate pairs. The first version still asks for a small JSON decision list; it produces one useful raw merge on an ambiguous disconnected-merge case, but independent validated calls can omit the required `decisions` field or emit non-JSON text. Table 7 shows the next design step: the model is restricted to one token, `KEEP`, `MERGE`, `SPLIT`, or `ABSTAIN`, for each proposed pair. The runtime then assembles the edit payload and applies the same schema, root/affected-service, evidence-reference, and fallback validators. This removes the free-form JSON failure mode in the three-hardcase probe: the validated action path records zero invalid actions, zero invalid-schema runs, zero fallbacks, and raises mean F1 from 0.6948 to 0.8857. On disconnected merge it reaches F1 1.0000 with three evidence-preserving merges; on temporal fragmentation it improves F1 from 0.5000 to 0.8000 but still over-reports; on ambiguous-overmerge it abstains and preserves the baseline. The evidence supports a sharper systems claim: reliable LLM-backed semantic reduction needs a constrained edit interface where the model supplies bounded semantic judgments and the system owns JSON construction, validation, and traceability.

The benchmark records more than a summary score: configuration, reducer name, injected and detected incidents, matched and missed incident IDs, evidence support, latency, token estimates, and failure classes. In the 2K live smoke, for example, three injected incidents had zero overlapping map candidates; the low F1 therefore diagnoses a coverage failure, not a reducer-quality result.

7.5 Lessons

The evaluation gives three lessons. First, the workload is not merely a classifier benchmark. Precision measures whether the reducer avoids inventing incidents from noisy evidence; recall measures whether the map and reduce stages preserve weak but real incidents; the missed-incident list shows which operator boundary failed. Second, the incident reducer improves quality mainly by changing the unit of output from windows to hypotheses. The output count drops toward the four injected incidents while recall remains high. Third, the map policy matters as much as the reducer: if a saturation event never becomes evidence, no downstream LLM can recover it without going back to raw telemetry.

These lessons define the role of an LLM reducer as a constrained semantic operator rather than a replacement for the pipeline. A model should not be rewarded for basic scanning, thresholding, or duplicate removal that deterministic operators already handle. Its useful role is to adjudicate ambiguous evidence groups, use service relationships to reject false positives, explain why symptoms belong together, and produce operator-facing reports from bounded evidence objects. The implemented online path exercises this contract and shows why validation is part of the system design: accepted model edits must preserve schema, evidence references, root/affected-service consistency, and cost visibility.

8 Open System Challenges

Semantic MapReduce deliberately separates the semantic control plane from the data and model-serving planes. Data engines still own storage access, scans, joins, stream windows, and distributed execution; model-serving engines still own

Table 7. One-token pairwise action probe on the same single-NPU endpoint. The three hard cases have full evidence coverage. Free-form pairwise reducers still fail the schema contract, while the action reducer restricts model output to an enum and lets the system assemble validated edits. This is a mechanism probe, not a multi-seed robustness claim.

Reducer	Precision	Recall	F1	Support recall	Reduce ms	Tokens	Edits	Invalid act.	Invalid schema	Fallback
Semantic graph	0.3810	0.3333	0.3463	0.4167	0.53	0.0	0.0	0.0	0	0.0
Hybrid hint	0.6349	0.9167	0.6948	0.6528	0.28	0.0	0.0	0.0	0	0.0
LLM pairwise	0.3810	0.3333	0.3463	0.4167	16313.58	506.7	0.0	0.0	3	1.0
Validated LLM pairwise	0.6349	0.9167	0.6948	0.6528	451.73	505.7	0.0	0.0	3	1.0
LLM pairwise action	0.8889	0.9167	0.8857	0.9444	327.17	534.0	2.3	0.0	0	0.0
Validated LLM pairwise action	0.8889	0.9167	0.8857	0.9444	325.71	534.0	2.3	0.0	0	0.0

inference. The orchestration layer owns a different contract: which observations become evidence, which reducer policy is applied, how hypotheses cite evidence, and how reducer behavior is traced. This scope exposes four concrete systems challenges rather than hiding them inside an agent prompt. First, evidence scope determines whether a reducer comparison is meaningful; the prototype addresses this with coverage sweeps, missed-incident classes, and baseline-aware evidence extraction. Second, LLM reducers can omit incidents, over-merge candidates, produce invalid structures, or cite incomplete evidence; the prototype addresses this with deterministic baselines, candidate-level editing, schema validation, unsafe-edit rejection, no-regression fallback, and one-token pairwise actions assembled by the system. Third, model calls add latency and token cost; the same report schema records these costs so reducer studies compare quality and overhead together. Fourth, data-plane integration is currently local; the next step is to attach Shard and MapEvidence to Spark, Flink, Ray, database, public AIOps, or production-derived telemetry sources while preserving the same evidence and trace contracts. These extensions should keep the evaluation order used here: coverage first, reducer quality second, live serving latency and token cost third.

9 Related Work

Large-scale data processing. MapReduce, Spark, and Flink provide the execution substrate for large-scale batch and stream analytics [2, 3, 19]. Semantic MapReduce borrows the local/global decomposition but changes the reducer contract from fixed aggregation to semantic evidence fusion.

Distributed AI execution. Ray provides a general distributed runtime for AI and Python applications [10]. The prototype can use such systems as execution substrates, but its focus is the explicit orchestration of LLM reasoning workflows and evidence traces.

LLM serving and microservice telemetry. vLLM shows how memory management and scheduling shape LLM serving performance [5]. DeathStarBench, FIRM, the Illinois trace dataset, OpenTelemetry Demo, AIOps Challenge 2020, and LO2 provide realistic service graphs, metrics, logs, traces, and fault labels [1, 4, 11, 12, 14, 15]. Our current workload is controlled rather than replayed from these datasets; they are candidates for replacing the generator while keeping the same evidence and reducer contract.

LLM-powered data processing. Recent data-management systems also make LLM calls into declarative or optimizable data-processing operators. LOTUS introduces semantic operators for bulk AI-based transformations over structured and unstructured data, while Palimpzest optimizes AI-powered analytical query plans across quality, latency, and cost [8, 13]. Semantic MapReduce is complementary: it focuses less on query optimization over document collections and more on workflow/stream orchestration, incident-level reduction, and evidence-linked audit traces over operational data.

LLM application frameworks. LangGraph, LlamaIndex, and AutoGen provide useful abstractions for agents, retrieval, tool use, and multi-agent workflows [6, 9, 17]. A full comparison requires verifying their dataflow, streaming, runtime, and audit semantics under the same workload. The narrower comparison here is that the prototype emphasizes workflow/stream/runtime orchestration with explicit semantic reduction and evidence provenance.

Data+AI platforms. Systems such as Databricks, Snowflake Cortex, and BigQuery ML integrate analytics engines with machine learning and LLM functionality. These platforms are important reference points, but they require systematic comparison before strong claims about relative capability. The prototype differs in emphasis: it is an open workflow/runtime layer that can sit above multiple engines and make reducer behavior auditable. The main comparison dimensions are APIs, execution boundaries, governance mechanisms, and traceability features.

Observability and incident management. Root-cause localization systems such as CloudRanger, MicroRank, and TraceRCA show that service dependencies, traces, and abnormal/normal execution contrast help diagnose microservice failures [7, 16, 18]. Semantic MapReduce does not replace these techniques; it asks how their outputs and other observability signals become auditable incident hypotheses.

Agentic data analysis. Agentic systems can write queries, call tools, inspect results, and refine answers. They are useful for exploration, but high-volume operational analysis also needs replayable control flow. This prototype chooses

explicit map stages, reducer stages, and evidence objects to improve evaluation and auditability.

Positioning summary. The closest intellectual ancestor of Semantic MapReduce is MapReduce itself: local processing followed by global reduction. The closest practical neighbors are stream processors, distributed AI runtimes, LLM workflow frameworks, RAG/data frameworks, and data+AI platforms. The prototype does not claim to be more scalable than Spark or Ray, more feature-complete than data+AI platforms, or more general than agent frameworks. Its narrower claim is that this workload needs a distinct structure: standard evidence operators, semantic reduction, evidence-linked reporting, and workflow traces under one reproducible contract.

10 Conclusion

Large-scale LLM-augmented analysis needs more than fast scans and more than chat over data. It needs a workflow layer that can partition observations, compress evidence, semantically reduce hypotheses, and preserve audit trails. Semantic MapReduce captures this need by extending the map/reduce structure from numeric aggregation to evidence fusion and explanation. The prototype's dataflow, stream, runtime, and serving boundaries make it a concrete substrate for this direction. The evaluation shows that incident-level semantic reduction is distinct from map-local alerts and window aggregation, and the live LLM experiments expose the key systems requirement: model edits must be coverage-gated, constrained to verifiable actions, schema-validated, evidence-linked, and reversible before they become trusted reducer output.

References

[1] Kirill Bakhtin, Simon Schneider, and Riccardo Scandariato. 2025. LO2: Microservice API Anomaly Dataset of Logs and Metrics. In *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering*.

[2] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* (2015).

[3] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*.

[4] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

[6] LangChain. 2026. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph. Accessed 2026-06-30.

[7] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, et al. 2021. Practical Root Cause Localization for Microservice Systems via Trace Analysis. In *Proceedings of the 29th IEEE/ACM International Symposium on Quality of Service*.

[8] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. arXiv:2405.14696 [cs.CL]

[9] LlamaIndex. 2026. LlamaIndex: Data Framework for LLM Applications. https://github.com/run-llama/llama_index. Accessed 2026-06-30.

[10] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*.

[11] NetManAIOps. 2020. AIOps Challenge 2020 Data. https://github.com/NetManAIOps/AIOps-Challenge-2020-Data. Accessed 2026-07-08.

[12] OpenTelemetry. 2026. OpenTelemetry Demo Architecture. https://opentelemetry.io/docs/demo/architecture/. Accessed 2026-07-08.

[13] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. arXiv:2407.11418 [cs.DB]

[14] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*.

[15] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. Pre-processed Tracing Data for Popular Microservice Benchmarks. Illinois Data Bank. https://databank.illinois.edu/datasets/IDB-6738796.

[16] Pengfei Wang, Jingmin Xu, Meng Ma, Wei Lin, Dan Pan, Yuan Wang, and Pengfei Chen. 2018. CloudRanger: Root Cause Identification for Cloud Native Systems. In *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*.

[17] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Adam White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]

[18] Guangba Yu, Pengfei Chen, Zicheng Li, Haifeng Zheng, Xusheng Lin, and Xiaofan Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *Proceedings of The Web Conference*.

[19] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation*.